

Environnement informatique

Présentation : Tom Niget

Mail de contact : polytech@niget.fr

Site et ressources : niget.fr

Cours réalisé grâce notamment aux ressources de :
Erick Gallesio, Stéphane Lavirotte, Pierre Hyvernât

Environnement informatique

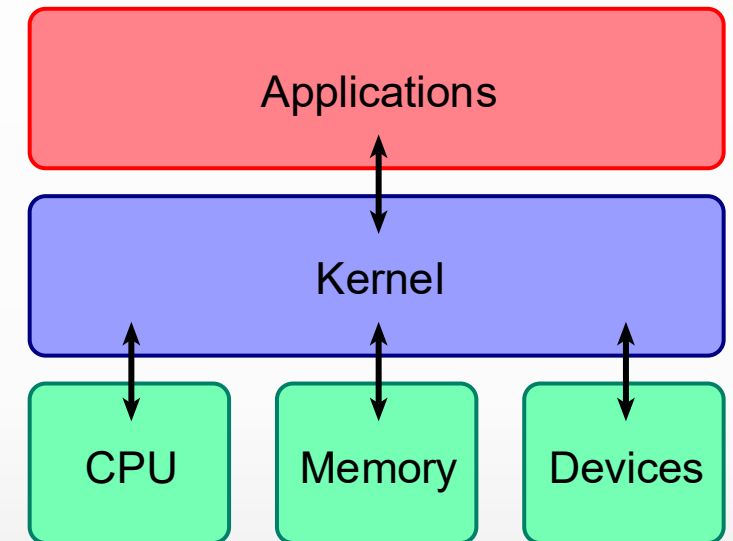
- Être confortable avec l'informatique comme **outil général**
- Comprendre la notion de **système d'exploitation** (« OS », Operating System)
 - Déjà abordé en Env. Info. en PeiP. Ça fera un rappel !
- Savoir **manipuler les OS courants** (famille Unix, Windows) et les technologies usuelles (réseau, Web, scripts)
 - Installation, commandes, fichiers, traitement de données et automatisation de tâches
- Maîtriser le **contrôle de version** (Git)

Environnement informatique

- Questions 1 à 4

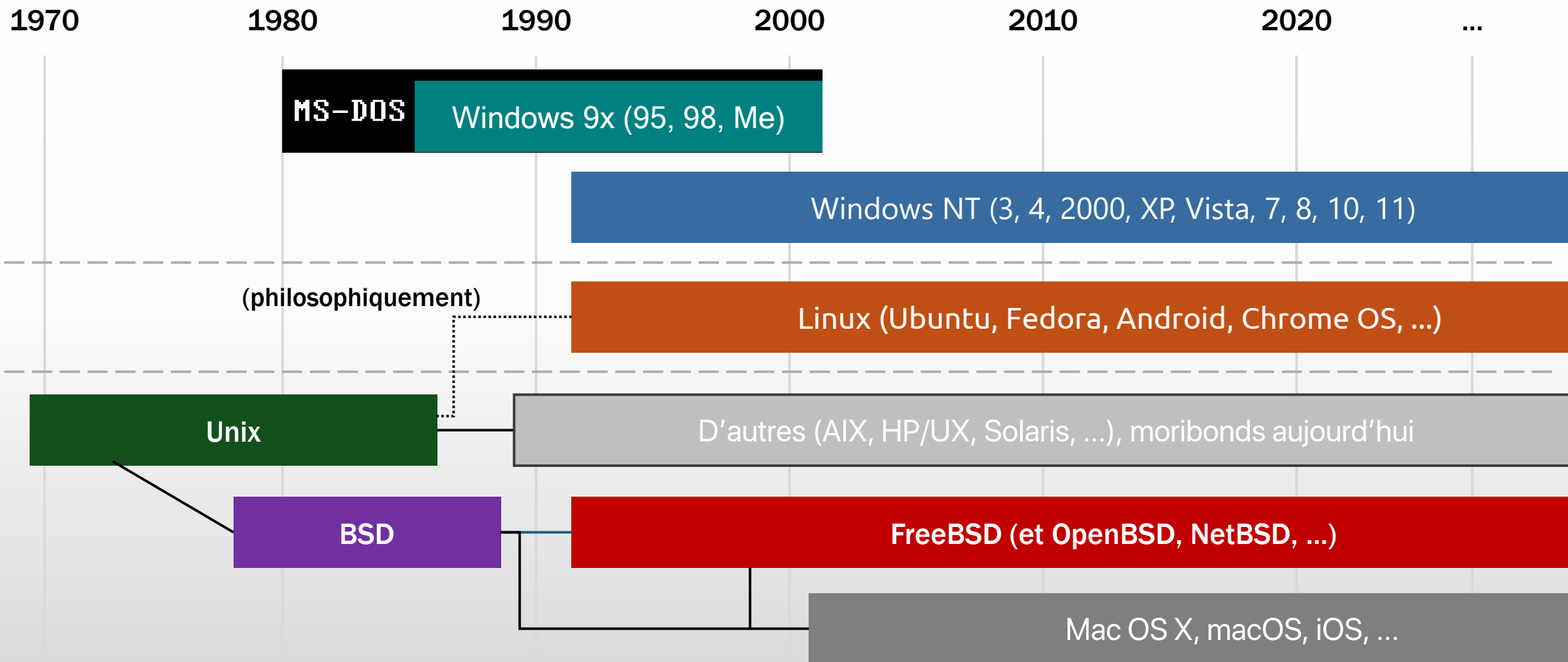
Systemes d'exploitation / OS

- OS : la couche logicielle entre l'utilisateur (et ses applications) et le matériel
- Classiquement, OS = noyau (kernel) + userland (utilitaires de base)
- Fournit des interfaces de programmation (API) aux applications

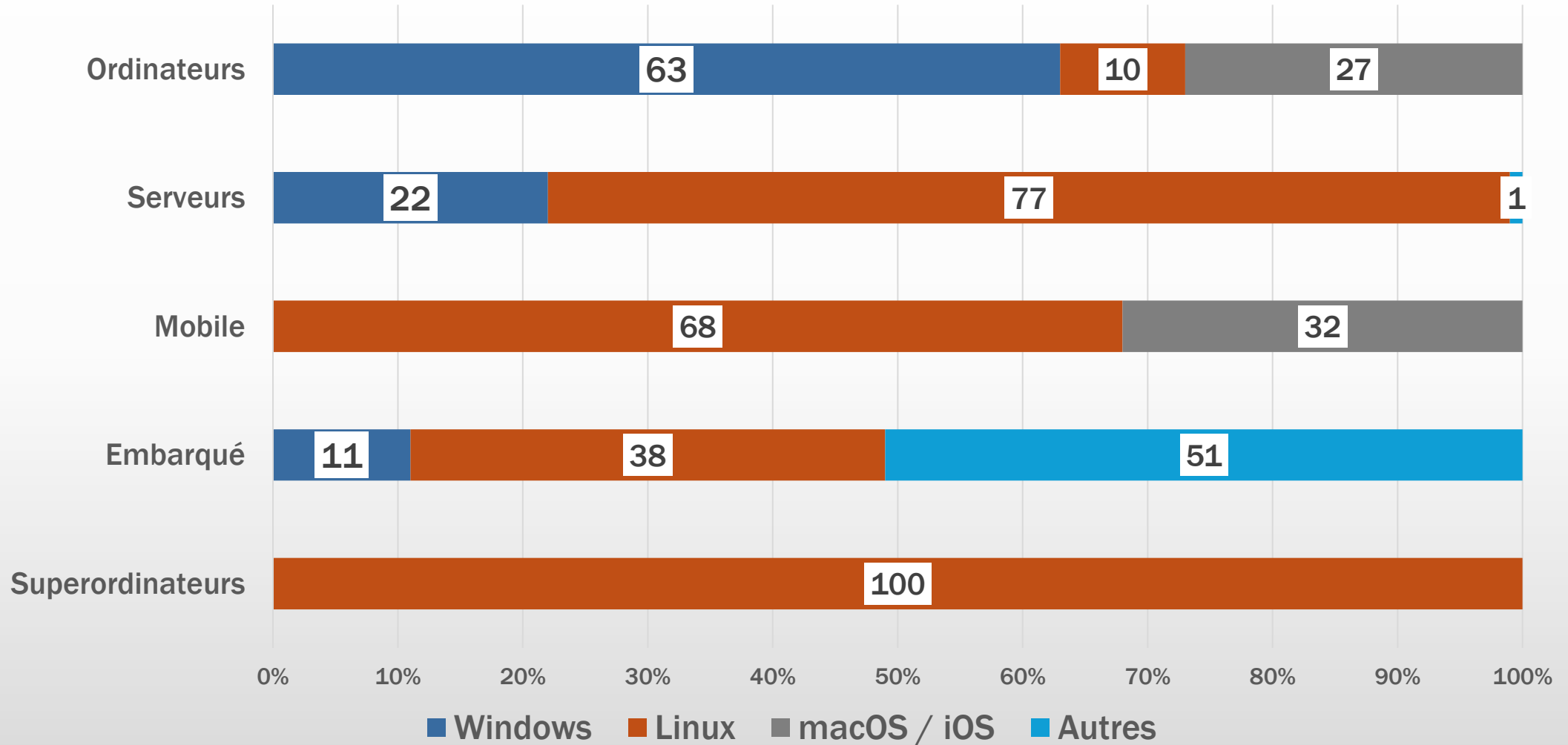


OS	Noyau
Windows	NT
Ubuntu, Fedora, Android, ChromeOS	Linux
macOS, iOS, iPadOS	XNU (Mach + BSD)

Généalogie des noyaux et OS courants



Parts de marché



Source : StatCounter 09/2026, TOP500

Systemes d'exploitation / OS

- Le noyau ne fait rien **tout seul** : c'est plutôt une sorte de **châssis** qui s'occupe de gérer la mémoire et les périphériques
- C'est un **programme comme les autres** : simplement il ne sert à rien sans un autre programme à lancer
- Exemples d'opérations (appels système) fournies par le noyau :
 - Lancer un programme
 - Lire / écrire des fichiers
 - Se connecter à une autre machine via le réseau

Systemes d'exploitation / OS

- Surtout : le noyau donne l'impression aux programmes d'avoir le monopole de la machine, c'est le **multi-tâche**
 - Chaque programme s'exécute **indépendamment** des autres
 - Le plantage d'un programme n'affecte que lui

Fichiers, codage, stockage

- Un ordinateur manipule des informations dont l'unité est l'octet
 - Octet : valeur numérique sur 8 bits (donc $2^8 = 256$ valeurs possibles)
- Un fichier a pour contenu une séquence d'octets ; c'est au programme qui « consomme » le fichier de donner un sens à ce contenu
 - Souvent, on donne une information sur le type de contenu à l'aide de l'**extension** du fichier : nomdefichier.**ext** (mais ce n'est qu'un indice non contractuel !)

Fichiers, codage, stockage

- Pour coder des informations sous forme d'octets, on définit des **encodages**.

Par exemple, pour le texte, on établit une liste de caractères, et on attribue à chacun un numéro. Exemple avec ASCII :

ASCII TABLE

Decimal	Hex	Char	Decimal	Hex	Char	Decimal	Hex	Char	Decimal	Hex	Char
0	0	[NULL]	32	20	[SPACE]	64	40	@	96	60	`
1	1	[START OF HEADING]	33	21	!	65	41	A	97	61	a
2	2	[START OF TEXT]	34	22	"	66	42	B	98	62	b
3	3	[END OF TEXT]	35	23	#	67	43	C	99	63	c
4	4	[END OF TRANSMISSION]	36	24	\$	68	44	D	100	64	d
5	5	[ENQUIRY]	37	25	%	69	45	E	101	65	e
6	6	[ACKNOWLEDGE]	38	26	&	70	46	F	102	66	f
7	7	[BELL]	39	27	'	71	47	G	103	67	g
8	8	[BACKSPACE]	40	28	(	72	48	H	104	68	h
9	9	[HORIZONTAL TAB]	41	29	)	73	49	I	105	69	i
10	A	[LINE FEED]	42	2A	*	74	4A	J	106	6A	j
11	B	[VERTICAL TAB]	43	2B	+	75	4B	K	107	6B	k
12	C	[FORM FEED]	44	2C	,	76	4C	L	108	6C	l
13	D	[CARRIAGE RETURN]	45	2D	-	77	4D	M	109	6D	m
14	E	[SHIFT OUT]	46	2E	.	78	4E	N	110	6E	n
15	F	[SHIFT IN]	47	2F	/	79	4F	O	111	6F	o
16	10	[DATA LINK ESCAPE]	48	30	0	80	50	P	112	70	p
17	11	[DEVICE CONTROL 1]	49	31	1	81	51	Q	113	71	q
18	12	[DEVICE CONTROL 2]	50	32	2	82	52	R	114	72	r
19	13	[DEVICE CONTROL 3]	51	33	3	83	53	S	115	73	s
20	14	[DEVICE CONTROL 4]	52	34	4	84	54	T	116	74	t
21	15	[NEGATIVE ACKNOWLEDGE]	53	35	5	85	55	U	117	75	u
22	16	[SYNCHRONOUS IDLE]	54	36	6	86	56	V	118	76	v
23	17	[END OF TRANS. BLOCK]	55	37	7	87	57	W	119	77	w
24	18	[CANCEL]	56	38	8	88	58	X	120	78	x
25	19	[END OF MEDIUM]	57	39	9	89	59	Y	121	79	y
26	1A	[SUBSTITUTE]	58	3A	:	90	5A	Z	122	7A	z
27	1B	[ESCAPE]	59	3B	;	91	5B	[	123	7B	{
28	1C	[FILE SEPARATOR]	60	3C	<	92	5C	\	124	7C	
29	1D	[GROUP SEPARATOR]	61	3D	=	93	5D	]	125	7D	}
30	1E	[RECORD SEPARATOR]	62	3E	>	94	5E	^	126	7E	~
31	1F	[UNIT SEPARATOR]	63	3F	?	95	5F	_	127	7F	[DEL]

Fichiers, codage, stockage

Fichier texte quelconque

Je m'appelle Alice et j'ai 20 ans. Je parle anglais, espagnol mais aussi français.

Trivial à lire (et écrire) pour un humain mais **très très dur à lire** pour une machine (il faut apprendre à la machine à parler français !).

Fichier texte structuré (ici, en JSON)

```
{  
  "name": "Alice",  
  "age": 20,  
  "spokenLanguages": ["English", "Spanish", "French"]  
}
```

Modérément facile à lire (et écrire) pour un humain, trivial à lire pour une machine, mais beaucoup de **perte d'espace**.

« nom suivi d'un octet nul, puis âge sur 1 octet, puis nombre de langues sur 1 octet, puis chaque nom de langue suivi d'un octet nul, puis un octet nul indiquant la fin du fichier »

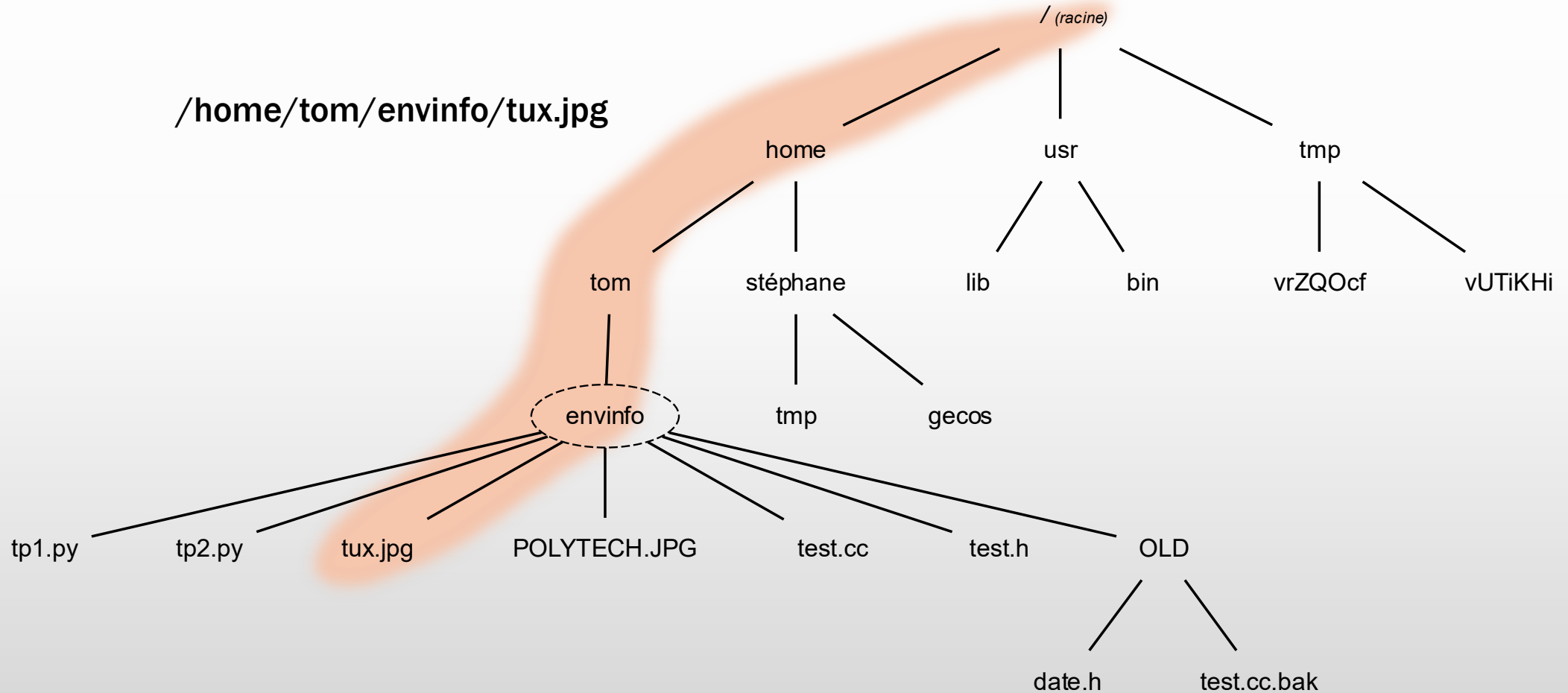
Fichier binaire (nécessairement structuré)

Offset(h)	00	01	02	03	04	05	06	07	08	09	0A	0B	0C	0D	0E	0F	Texte Décodé
00000000	41	6C	69	63	65	00	14	03	45	6E	67	6C	69	73	68	00	Alice...English.
00000010	53	70	61	6E	69	73	68	00	46	72	65	6E	63	68	00	00	Spanish.French..

Difficile voire impossible à comprendre si on ne connaît pas à l'avance la structure ; facile d'écrire un programme décodant la structure, très économe en espace.

Fichiers, codage, stockage

- Les fichiers existent dans une arborescence de dossiers :



Fichiers, codage, stockage

- On note les chemins avec un / (ou un \ sous Windows) pour séparer les éléments
- Un chemin peut être absolu (partir de la racine) ou relatif
 - Sous Unix, la racine est appelée /
 - Sous Windows, il s'agira de C:\, D:\, etc
 - Un chemin relatif dépendra du dossier courant (working directory) du programme
- .. est un chemin spécial qui désigne le dossier parent. Donc /a/b/c/.. revient à écrire /a/b
 - Sauf dans certains cas particuliers...
- . est un autre chemin spécial qui désigne le dossier en cours

Travail en ligne de commande

- L'interface la plus primitive pour exécuter des programmes et interagir avec le système
- Historiquement, la seule, mais toujours essentielle aujourd'hui pour aller plus loin que l'interface graphique et automatiser
- Pour un usage interactif, il faut un **shell** : programme qui va permettre d'exécuter des commandes à la volée
 - COMMAND et PowerShell (Windows)
 - Bash, Zsh, Csh (Unix)

Travail en ligne de commande (sous Unix)

- Instruction en ligne de commande : simplement du texte
- Une commande peut souvent prendre des **paramètres**, reconnaissables à leurs tirets : `ls -l --color dossier`
- Les commandes sont simplement des **programmes** livrés de base avec l'OS
 - Une bonne partie des commandes de base correspondent directement à un appel système précis : `chmod`, `rm`, `id`, ...

Fichiers, codage, stockage

- Les motifs permettent de traiter un ensemble de fichiers via un seul chemin
- Deux motifs élémentaires :
 - * : matche n'importe quoi
 - ? : matche n'importe quel caractère
- « Hello* » matche n'importe quelle chaîne commençant par Hello : « Hello », « Helloabc », « Hello123 »
- Motif composite :
 - [abc] matche soit a, soit b, soit c
 - [0-9] matche de 0 à 9, [A-F] matche les lettres de A à F (majuscule)
 - [*] matche le caractère étoile

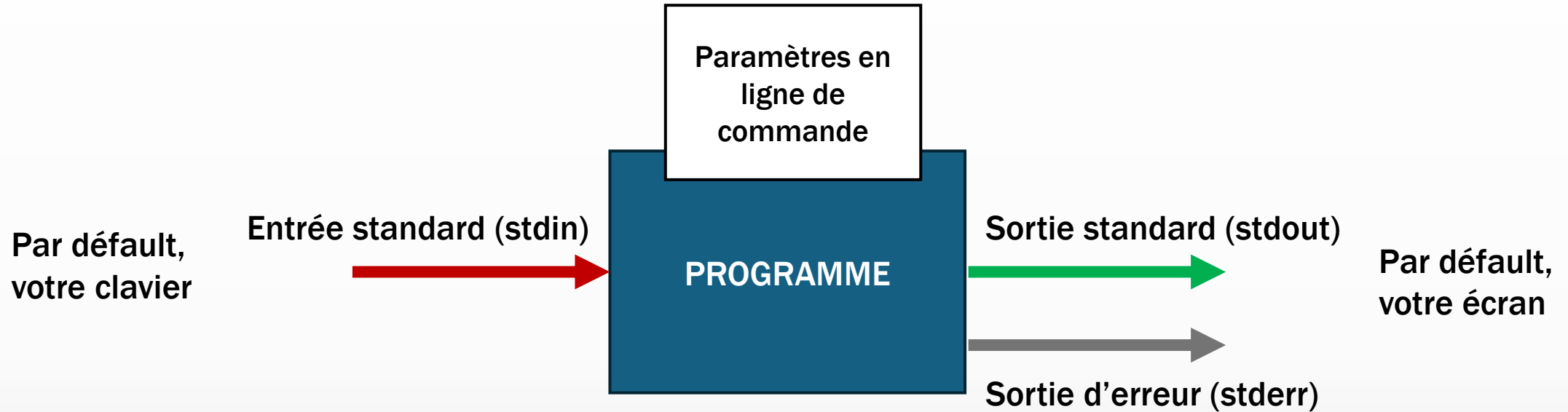
Quelques commandes Unix courantes

- **ls** : lister le contenu d'un dossier
- **cp** : copier
- **rm** : supprimer
- **mv** : déplacer
- **man** : obtenir l'aide d'une commande
- **nano** : éditeur simple dans le terminal
- **cat** : afficher le contenu d'un fichier
- **echo** : afficher un texte
- **grep** : chercher dans du texte

Exemple

```
$ pwd          # affiche le dossier courant (print working directory)
/home/tom
$ ls           # liste le contenu du dossier
truc bidule chose
$ cat truc     # affiche le contenu du fichier
Bonjour tout le monde !
Au revoir !
$ rm truc      # supprime (remove) le fichier
$ ls
bidule chose
```

Travail en ligne de commande



- Configuration via les **paramètres** en ligne de commande
- Passage de **données** via les canaux standard

Combinaison de commandes



- On peut enchaîner plusieurs commandes, passant la sortie de l'une à l'entrée de l'autre :
- `grep abc` : filtre les lignes de l'entrée qui contiennent `abc`, et affiche le résultat en sortie
- `ls` : liste les éléments du dossier
- `ls | grep abc` : liste les éléments du dossier dont le nom contient `abc`

Exercice

```
$ man sort
```

```
sort - sort lines of text files
```

```
-r, --reverse
```

```
reverse the result of comparisons
```

```
$ man grep
```

```
grep - print lines that match patterns
```

```
-v, --invert-match
```

```
Invert the sense of matching, to select non-matching lines.
```

```
$ man head
```

```
head - output the first part
```

```
Print the first 10 lines to standard output
```

```
$ ls | grep -v truc | sort -r | head
```